

The 32 MB number is misleading here. R is failing because the session is already near its memory ceiling after trying to read 4.63 GB of compressed `acc_long` partitions, which may expand to many times that size in RAM.

Thought for 50s

Goal

Resolve the error:

```
Error: cannot allocate vector of size 31.7 Mb
```

when running:

```
acc_long <- load_metadata_table("acc_long")
```

Problem

The problem is **not** that 31.7 MB is large. The problem is that R is already near its memory limit or has fragmented memory after trying to load:

```
Reading 81 acc_long partition(s); compressed input size = 4.63 GB.
```

The existing loader tries to read all 81 `acc_long` partitions into a list and then bind them. That creates a large peak memory burden because R temporarily holds:

```
all individual acc_long partitions  
+ the list of partitions  
+ the combined acc_long object  
+ internal copy vectors
```

The original `metadata_prep.R` creates `all_wide` and then expands it to `acc_long` with `pivot_longer()`, so `acc_long` is the largest metadata table.

For Analysis A, you do **not** need full `acc_long`. You only need `acc_mv`, because `code-A.R` fits:

```
manova(cbind(rmse, L1, Linf) ~ ..., data = acc_mv)
```

Best solution

Do **not** load full `acc_long`.

Instead, build `acc_mv` directly from the partitioned `all_wide` files. This avoids the full `acc_long` table and should reduce memory use by about a factor of 3 or more.

Exact R syntax

Start a **fresh R session** first. If you are still at:

```
Browse[1]>
```

type:

```
Q
```

Then restart R.

Run this:

```
rm(list = ls())  
gc()  
  
setwd("C:/Users/Jon.Brodziak/Desktop/Dirichlet Study/hake/RUN1")  
  
library(data.table)  
  
out_dir <- file.path(getwd(), "metadata_output")  
  
wide_files <- list.files(  
  file.path(out_dir, "all_wide_parts"),  
  pattern = "^all_wide_part_[0-9]+\\.rds$",  
  full.names = TRUE  
)
```

Then build `acc_mv` partitions directly from `all_wide`:

```
acc_mv_dir <- file.path(out_dir, "acc_mv_parts")

if (dir.exists(acc_mv_dir)) {
  unlink(acc_mv_dir, recursive = TRUE, force = TRUE)
}

dir.create(acc_mv_dir, recursive = TRUE)

method_levels <- c("i", "ii", "iii", "iv", "v")

id_cols <- c(
  "example_id", "mesh_id", "sim_id", "dataset_id", "simplex_id",
  "p1", "p2", "p3",
  "design_block", "G", "theta_true", "theta_CV", "sigma",
  "mean_nsamp", "ln_sd", "nb_size", "N_range_label"
)

manifest <- vector("list", length(wide_files))

for (k in seq_along(wide_files)) {

  message("Processing acc_mv partition ", k, " of ", length(wide_files))

  wide <- readRDS(wide_files[k])
  setDT(wide)

  keep_id_cols <- intersect(id_cols, names(wide))

  pieces <- lapply(method_levels, function(m) {

    rmse_col <- paste0("rmse_", m)
    L1_col   <- paste0("L1_norm_", m)
    Linf_col <- paste0("Linf_norm_", m)

    missing_cols <- setdiff(c(rmse_col, L1_col, Linf_col), names(wide))

    if (length(missing_cols) > 0) {
      stop("Missing columns in ", basename(wide_files[k]), ": ",
        paste(missing_cols, collapse = ", "))
    }

    data.table(
      wide[, ..keep_id_cols],
```

```

    method = factor(m, levels = method_levels),
    rmse     = wide[[rmse_col]],
    L1       = wide[[L1_col]],
    Linf     = wide[[Linf_col]]
  )
})

acc_mv_part <- rbindlist(pieces, use.names = TRUE, fill = TRUE)

out_file <- file.path(acc_mv_dir, sprintf("acc_mv_part_%03d.rds", k))

saveRDS(acc_mv_part, out_file, compress = "gzip")

manifest[[k]] <- data.table(
  part_id = k,
  input_file = wide_files[k],
  output_file = out_file,
  nrow_acc_mv = nrow(acc_mv_part),
  object_gb = as.numeric(object.size(acc_mv_part)) / 1024^3
)

rm(wide, pieces, acc_mv_part)
gc()
}

acc_mv_manifest <- rbindlist(manifest)

fwrite(
  acc_mv_manifest,
  file.path(out_dir, "acc_mv_manifest.csv")
)

sum(acc_mv_manifest$nrow_acc_mv)
max(acc_mv_manifest$object_gb)

```

Now load only the smaller `acc_mv` table:

```

rm(list = setdiff(ls(), c("out_dir")))
gc()

library(data.table)

acc_mv_files <- list.files(
  file.path(out_dir, "acc_mv_parts"),
  pattern = "^acc_mv_part_[0-9]+\\.rds$",
  full.names = TRUE
)

```

```
)

sum(file.info(acc_mv_files)$size) / 1024^3

acc_mv <- rbindlist(
  lapply(acc_mv_files, readRDS),
  use.names = TRUE,
  fill = TRUE
)

nrow(acc_mv)
format(object.size(acc_mv), units = "GB")
gc()
```

Then run Analysis A:

```
source("code-A.R")
```

If loading all `acc_mv` still fails

Then run a smaller diagnostic load first:

```
acc_mv_test <- rbindlist(
  lapply(acc_mv_files[1:5], readRDS),
  use.names = TRUE,
  fill = TRUE
)

nrow(acc_mv_test)
format(object.size(acc_mv_test), units = "GB")

rm(acc_mv_test)
gc()
```

Estimate the full size:

```
test_size_gb <- as.numeric(object.size(
  rbindlist(lapply(acc_mv_files[1:5], readRDS),
    use.names = TRUE, fill = TRUE)
```

```

)) / 1024^3

estimated_full_gb <- test_size_gb * length(acc_mv_files) / 5

estimated_full_gb

```

If estimated full `acc_mv` is too large for memory, then the full MANOVA in Analysis A may also be too large, because `manova()` must build an internal model matrix from the full `acc_mv` object.

Why this solves the 32 MB error

This avoids the failing command:

```
acc_long <- load_metadata_table("acc_long")
```

and replaces it with a smaller object:

```
acc_mv
```

That is valid for Analysis A because Analysis A uses only the three multivariate accuracy responses:

```
rmse, L1, Linf
```

by method and design covariates. It does not require the full long table with one row per metric.

Bottom line

Do not try to load `acc_long` for Analysis A. Build `acc_mv` directly from `all_wide_parts`, write it to `metadata_output/acc_mv_parts`, and then load only `acc_mv`. This removes the largest avoidable memory burden and directly matches the object required by `code-A.R`.